

ニューラルネットワーク

ニューラルネットワーク



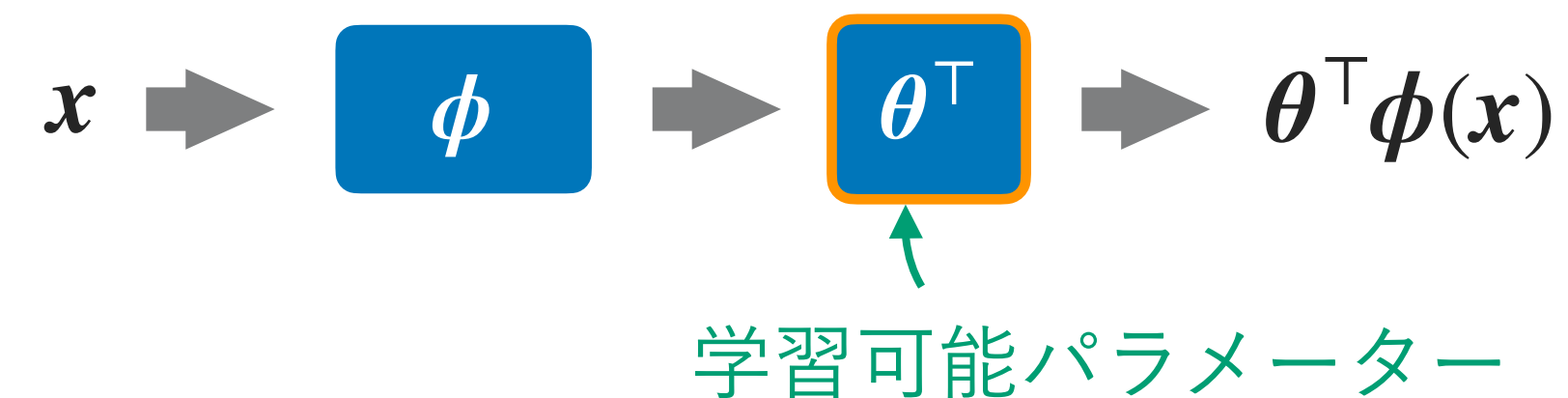
次は、ニューラルネットワークとよばれる柔軟なモデルを解説します

- ニューラルネットワークというモデルは、非線形モデルの主要なものの一つであり、実用的にも広く使われている。
- どういう学習の仕組みになっているかを理解しておくと、ここまでの内容と、近年のニューラルネットワークを用いた高度な機械学習モデルに関する話がつながって、裏の仕組みがイメージしやすくなる

ニューラルネット > 複雑なモデルと特徴量

ニューラルネットのポイントは、変換の階層性

- 線型モデルでは、出力値へのパラメーターの関与は、出力直前の一箇所だけ



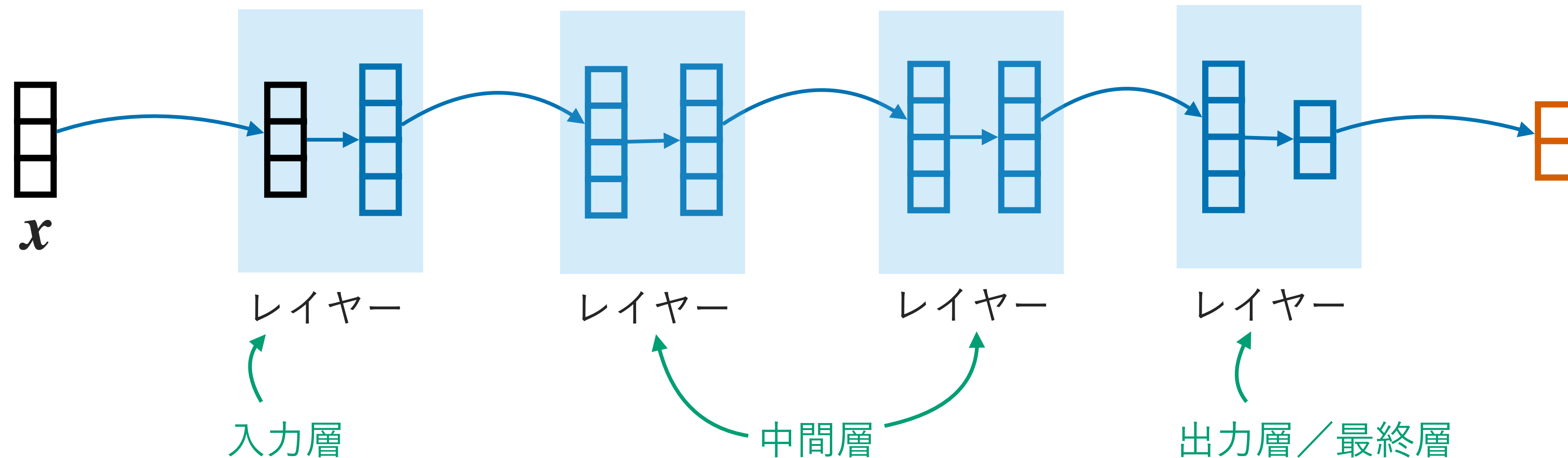
- ニューラルネットワーク (neural network, NN) では……



ニューラルネット > 基本の構成要素 > 順伝播型NN

最も基本的なニューラルネットワークの構造

- 順伝播型ニューラルネットワーク (Feed-forward neural network, FNN)
 - レイヤー (layer, 層) とよばれる部品を直列に繋げて複雑な関数を作る



【用語】 特別な味付けがないという比喻で、「バニラ」ニューラルネットワーク (vanilla neural network) とも呼ばれる。多層パーセプトロン (Multi-layer Perceptron, MLP) とも呼ばれる。伝播は、伝搬 (でんぱん) ではなく伝播 (でんぱ) であることに注意。

講義の前提知識＞ノテーションの注意

この講義では、1変数関数は断りなく多次元のベクトルに適用できるとする

- 多次元の出力をするモデルを扱う上で、便利な記号を用意しておく.
- ベクトル $\boldsymbol{a}$ に対して、1変数関数 ψ を成分ごとに適用したものを $\psi(\boldsymbol{a})$ で表す.

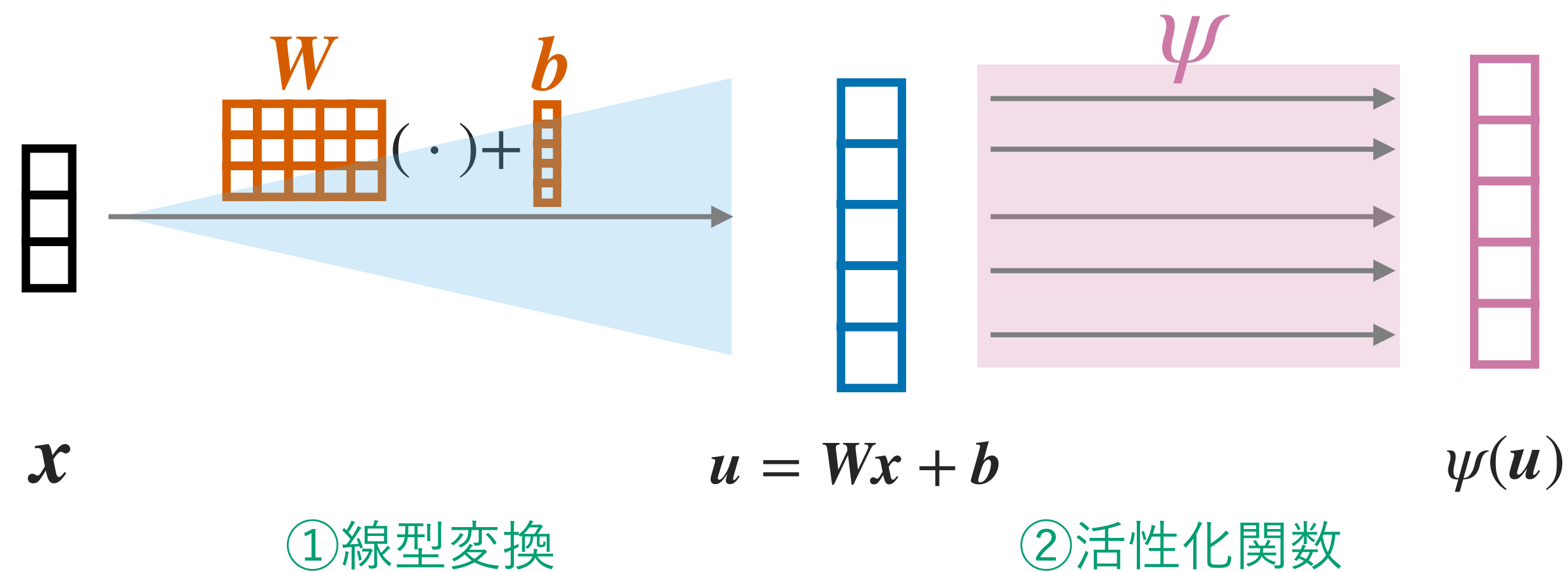
$$\psi(\boldsymbol{a}) := \begin{bmatrix} \psi(a_1) \\ \psi(a_2) \\ \vdots \\ \psi(a_d) \end{bmatrix}$$

ニューラルネット > 基本の構成要素 > レイヤー

各レイヤーの中身は、「線型変換を施してから活性化関数」でできている

■ 各レイヤー内の計算は $x \mapsto \psi(Wx + b)$ (W, b はパラメーター, ψ は固定)

- 線型変換 $x \mapsto Wx + b$ の後, 共通の ψ を成分毎に適用 $\psi(u) = [\psi(u_1), \dots, \psi(u_r)]^\top$
- W は重み, b はバイアス, ψ は活性化関数 (activation function) という



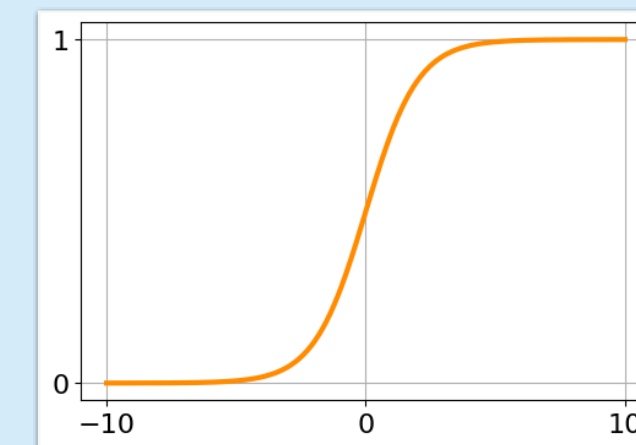
ニューラルネット > 基本の構成要素 > 活性化関数

よく使われる活性化関数（他にも多数提案されている）

Sigmoid

（標準ロジスティック関数）

$$\psi(u) = 1/(1 + \exp(-u))$$

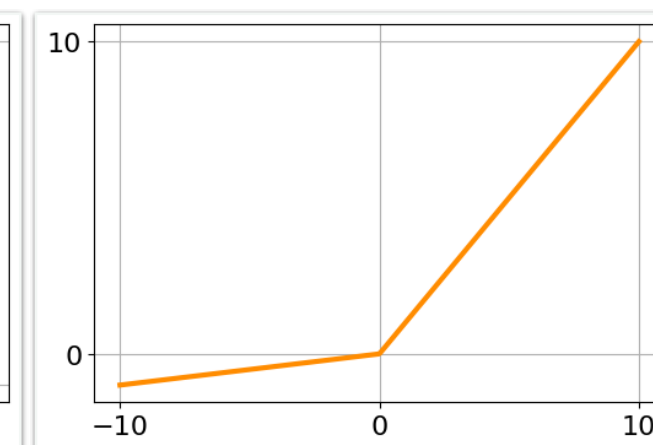
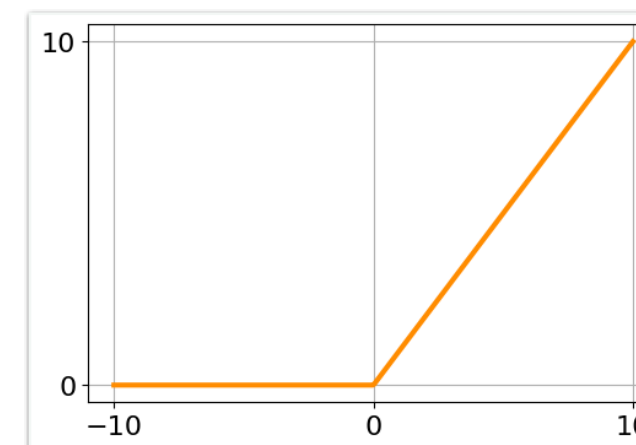


ReLU

$$\psi(u) = \max\{0, u\}$$

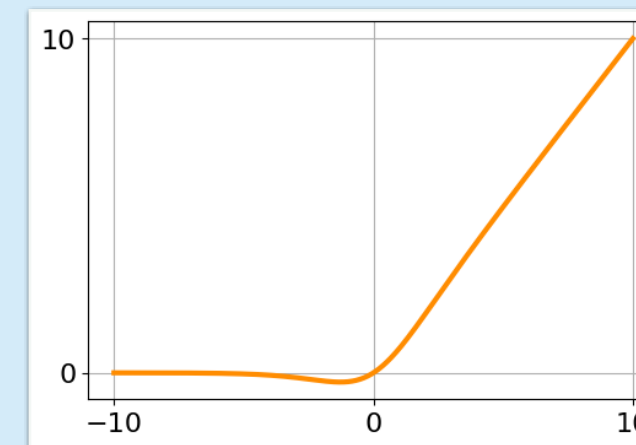
Leaky ReLU

$$\psi(u) = \max\{\alpha u, u\} \quad (0 \leq \alpha < 1)$$



Swish

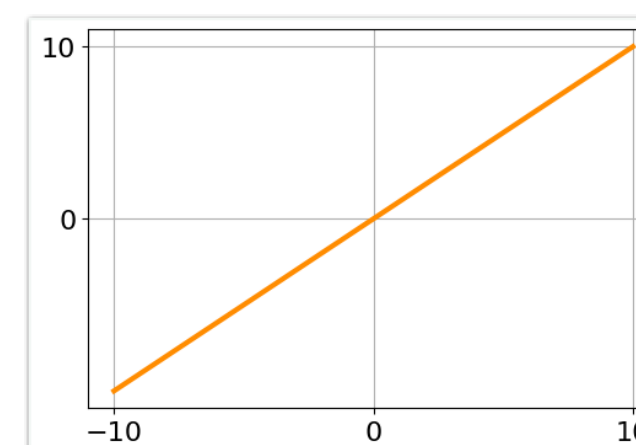
$$\psi(u) = u/(1 + \exp(-\beta u))$$



Linear activation

$$\psi(u) = u$$

（恒等写像）

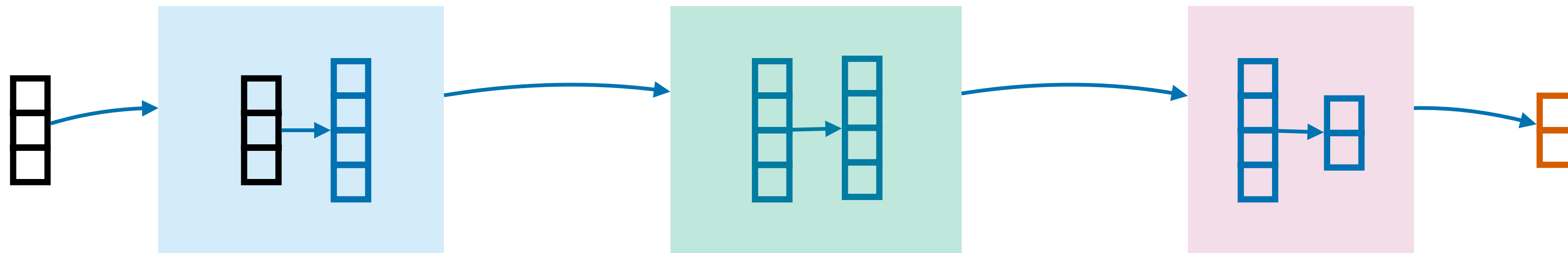


ニューラルネット > 基本の構成要素 > 順伝播型NN

このモデルは階層性がポイント

- 一つ一つのレイヤーは、さほど複雑な処理は行わない
 - 線型変換 + 成分ごとの活性化関数
 - しかし、それを多数重ねることで、全体としては複雑な関数を表現

$$\mathbf{x} \mapsto \psi_1(\mathbf{W}_1(\mathbf{x}) + \mathbf{b}_1) \mapsto \psi_2(\mathbf{W}_2(\psi_1(\mathbf{W}_1(\mathbf{x}) + \mathbf{b}_1) + \mathbf{b}_2)) \mapsto \psi_3(\mathbf{W}_3(\dots$$

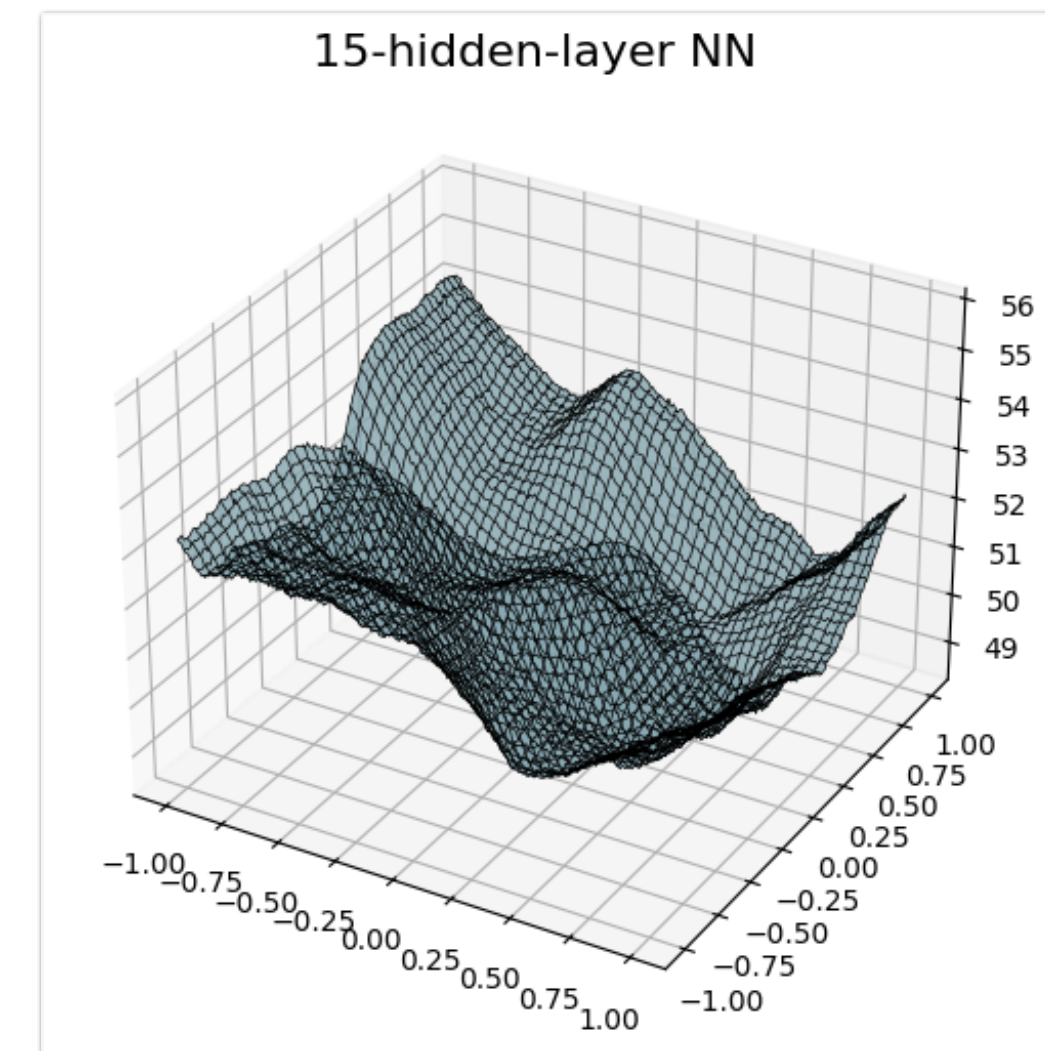
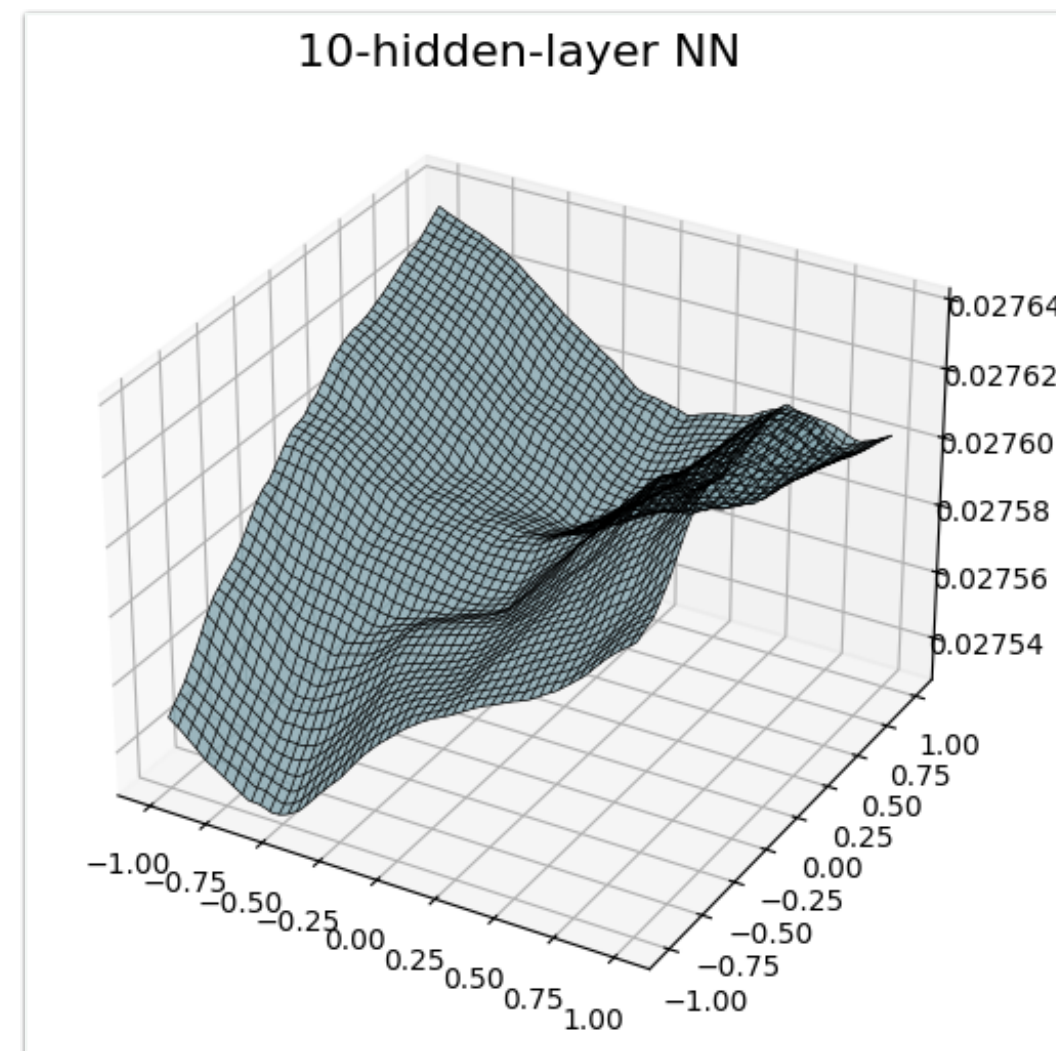
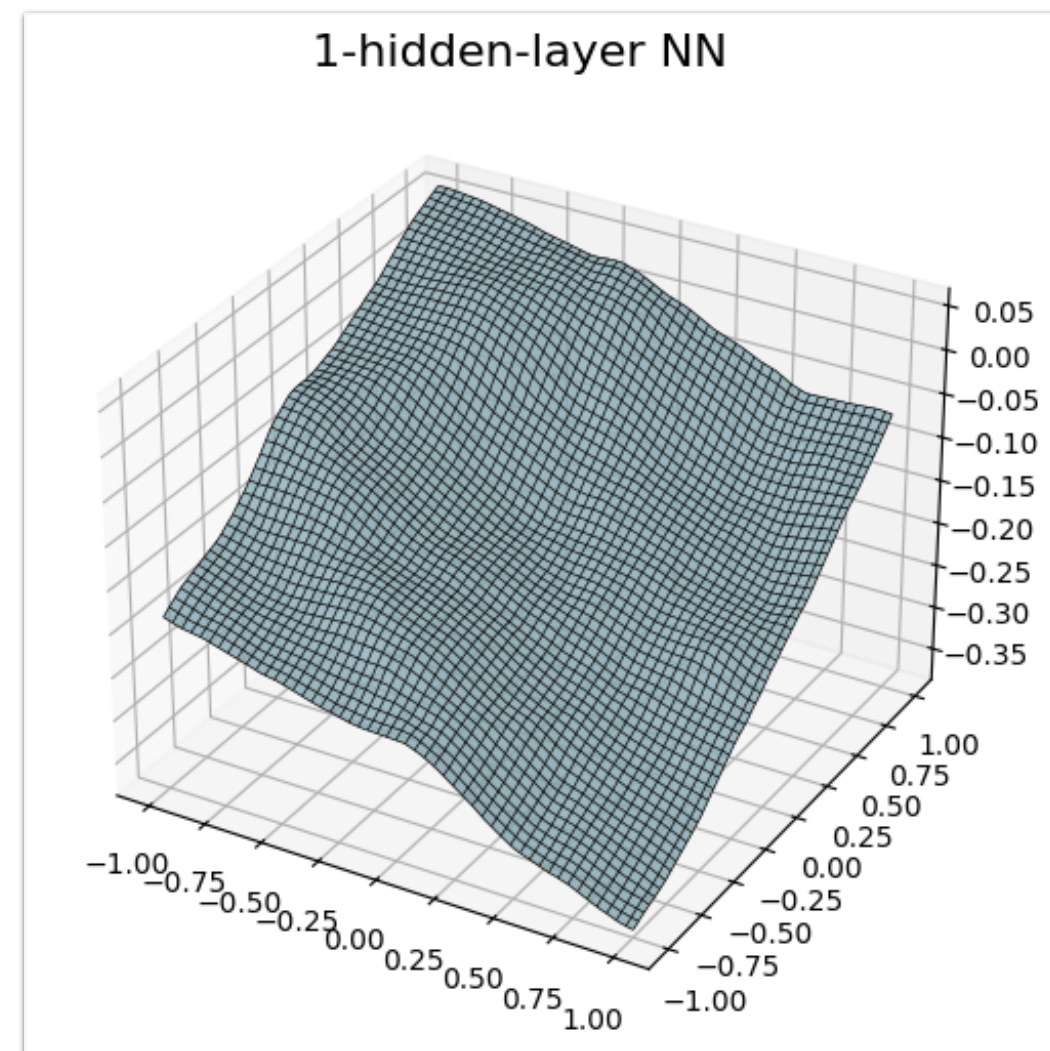


- 各レイヤー内の処理には調整可能なパラメーター ($\mathbf{W}, \mathbf{b}$) が付いている

ニューラルネット＞基本の構成要素

隠れ層を増やしていくと，関数として複雑化していく

- 既存の層は変えずに，ランダム初期化した層を順次追加していった場合

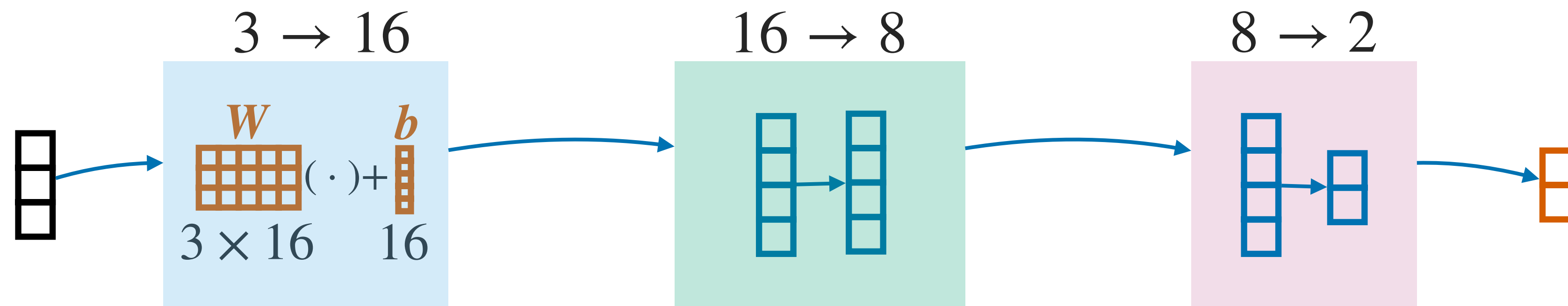


レイヤーの数

ニューラルネット＞基本の構成要素＞順伝播型NN

パラメーターの形状制約：隣接層は入出力の次元が合致する必要がある

- 入力層・出力層のパラメーターは、モデルとしての入出力次元に合わせる
- 中間層の出力の次元数は任意に選択可能（ハイパーパラメーター）
 - 例）3次元入力→**16次元**→**8次元**→2次元出力
 - 隣り合うレイヤー間では、出力を受け渡せる必要がある



【注意】 入力層・出力層の次元数は、受け付ける入力次元数・欲しい出力次元数に合わせる必要がある。

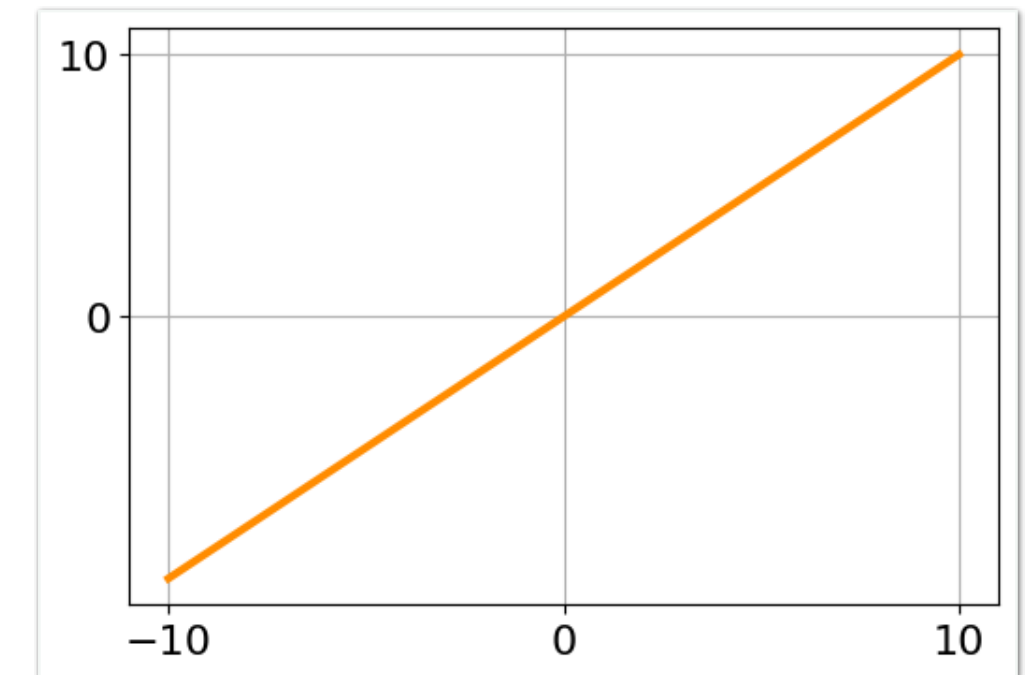
ニューラルネットワーク > 基本の構成要素 > 出力層

出力層の活性化関数は，用途に応じて選ぶ

■ 回帰の場合

- 実数出力にしたいので恒等活性化関数

$$\psi(u) = u$$



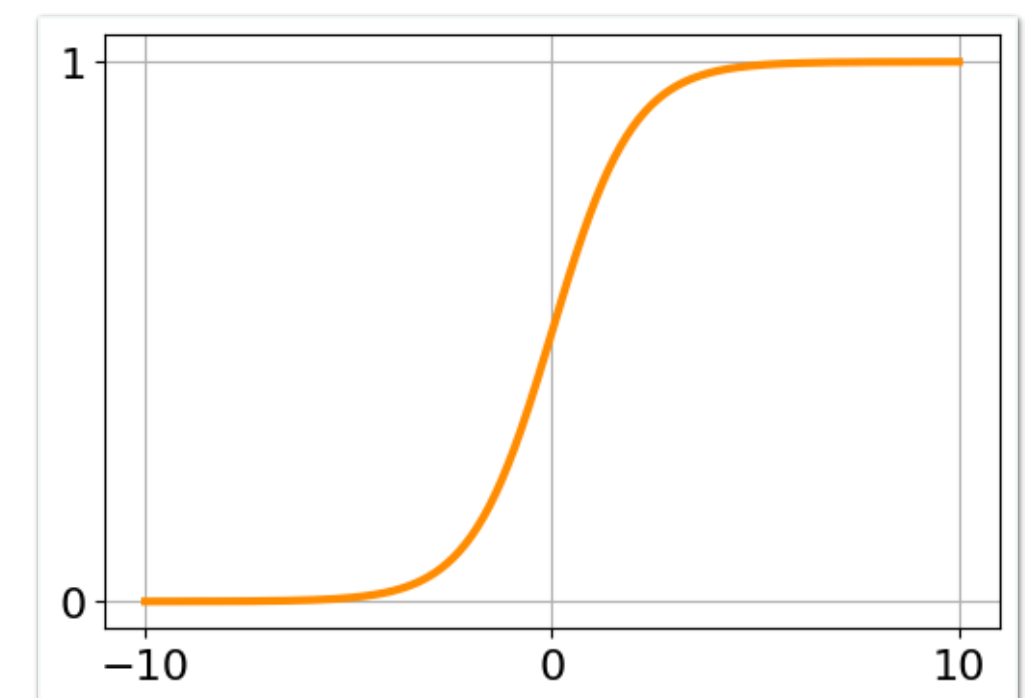
■ 分類の場合

- 2クラス確率論的分類 → シグモイド関数（ロジスティック関数）

$$\psi(u) = \text{Sigmoid}(u)$$

- 多クラス確率論的分類 → ソフトマックス関数

$$\psi(u) = \text{Softmax}(u)$$



【補足】 ソフトマックスのように，1変数関数ではないものも活性化関数と呼ぶ。

ニューラルネットワーク



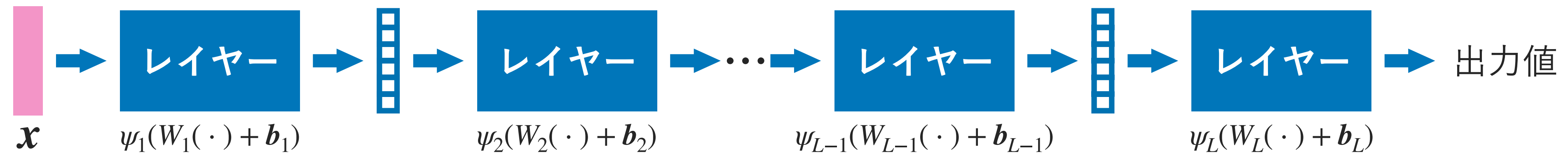
ニューラルネットの基本はレイヤーという部品

- レイヤー（層）という部品を積み重ねて複雑な関数を作る
 - 1つ1つの層は，入力を出力に変換する比較的簡単なパラメーター付きの関数
 - 1つ1つの層は簡単でも，いくつも合成すると複雑なものを表せる

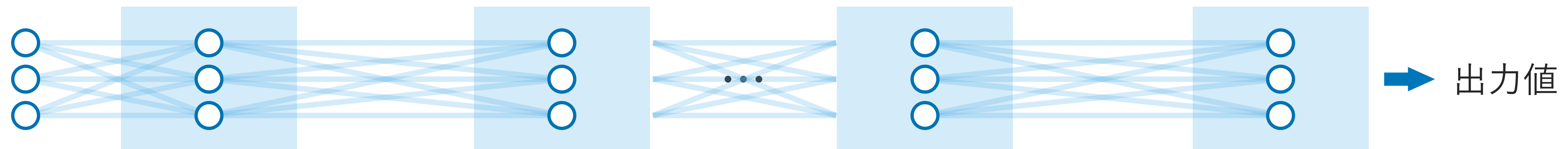
ニューラルネット > モデルの構造

ニューラルネットは2通りの見方がある（レイヤーとユニット）

- ここまでの見方：多数のレイヤーという変換を積み重ねたもの



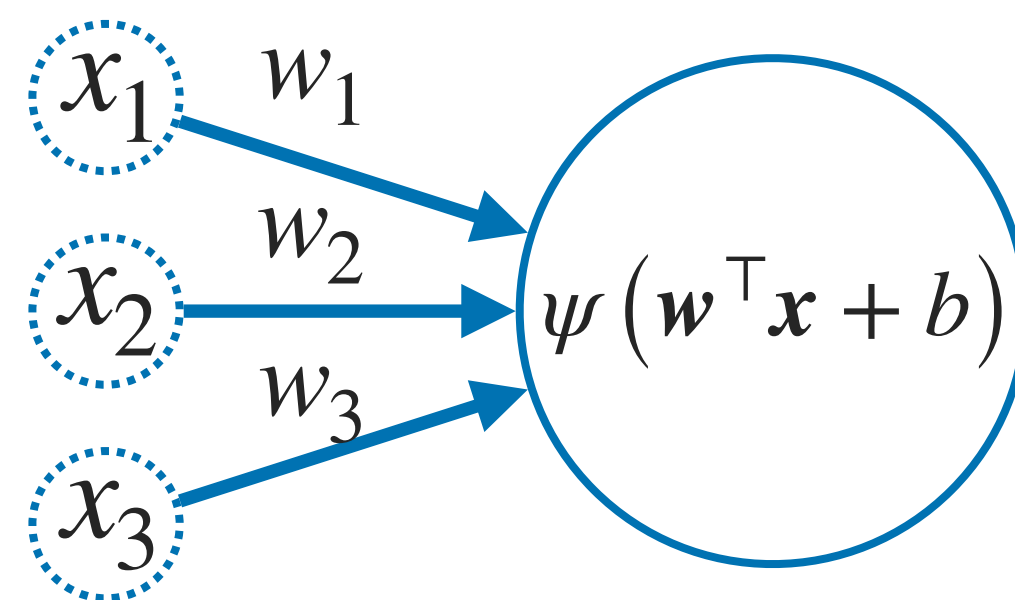
- 別の見方：多数のユニット（unit, 素子）を繋いだもの



ニューラルネット＞表現図式と用語＞ユニット

1個1個のユニットの仕組み

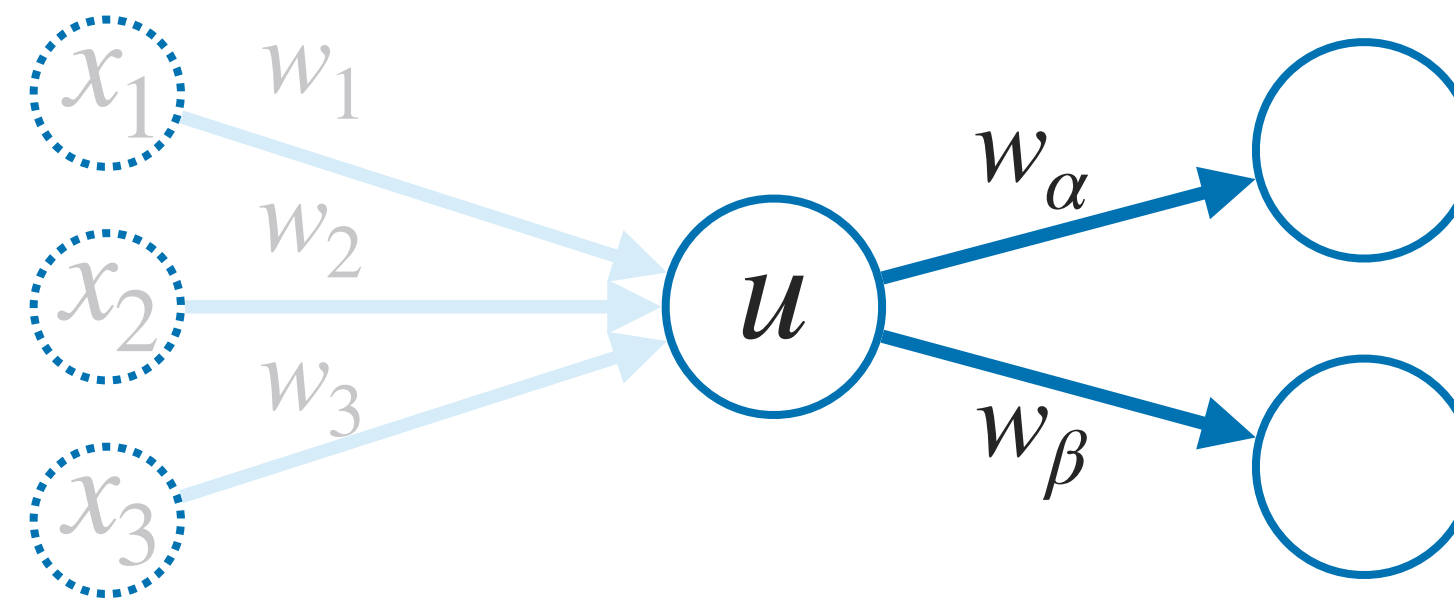
- まずは入力を受け取って、自分の値を計算して保持する
 1. 入力側のユニットたちから入力ベクトル $\mathbf{x} = (x_1, \dots, x_d)^\top$ を受け取る
 2. **エッジ**（繋いだ線）の重み $\mathbf{w} = (w_1, \dots, w_d)^\top$ とバイアス b で $\mathbf{w}^\top \mathbf{x} + b$ を計算
 3. 活性化関数を施した $u = \psi(\mathbf{w}^\top \mathbf{x} + b)$ が現在の値（**活性値：activation**）



ニューラルネット＞表現図式と用語＞ユニット

1個1個のユニットの仕組み

- そして、次は自分の活性値を、繋がっている出力側のユニットに受け渡す



ユニットが計算結果の値を保持
→ 他のユニットの入力になる

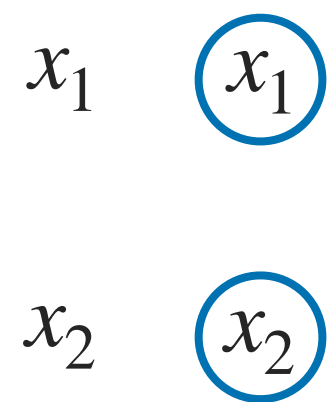
- ユニットは次で表される小さなモデルともいえる
(重み w とバイアス b がパラメーター，活性化関数 ψ は事前に決定)

$$\mathbf{x} \mapsto \psi(\mathbf{w}^\top \mathbf{x} + b)$$

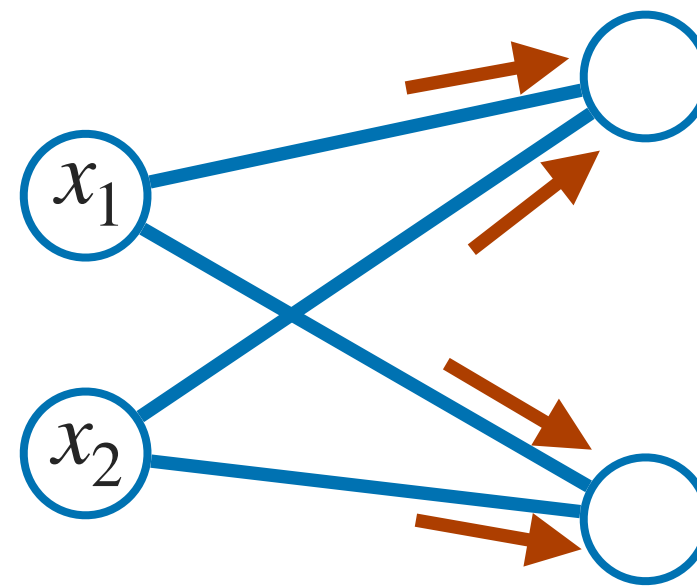
ニューラルネット＞モデルの構造

ユニットという見方で説明すると，ニューラルネットの計算の流れは……

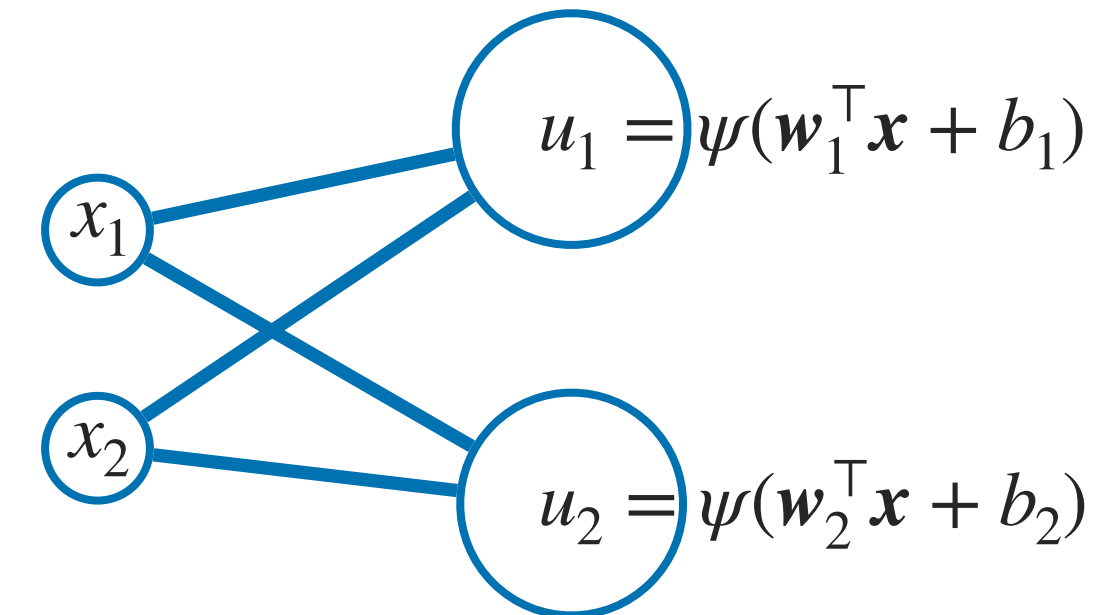
- ユニットは自分の活性値を保持し，次のユニットに値を渡す



まず入力特徴量の値が
入力層のユニットに
そのままセットされる



ユニットの現在の値は
後続のユニットへ渡される



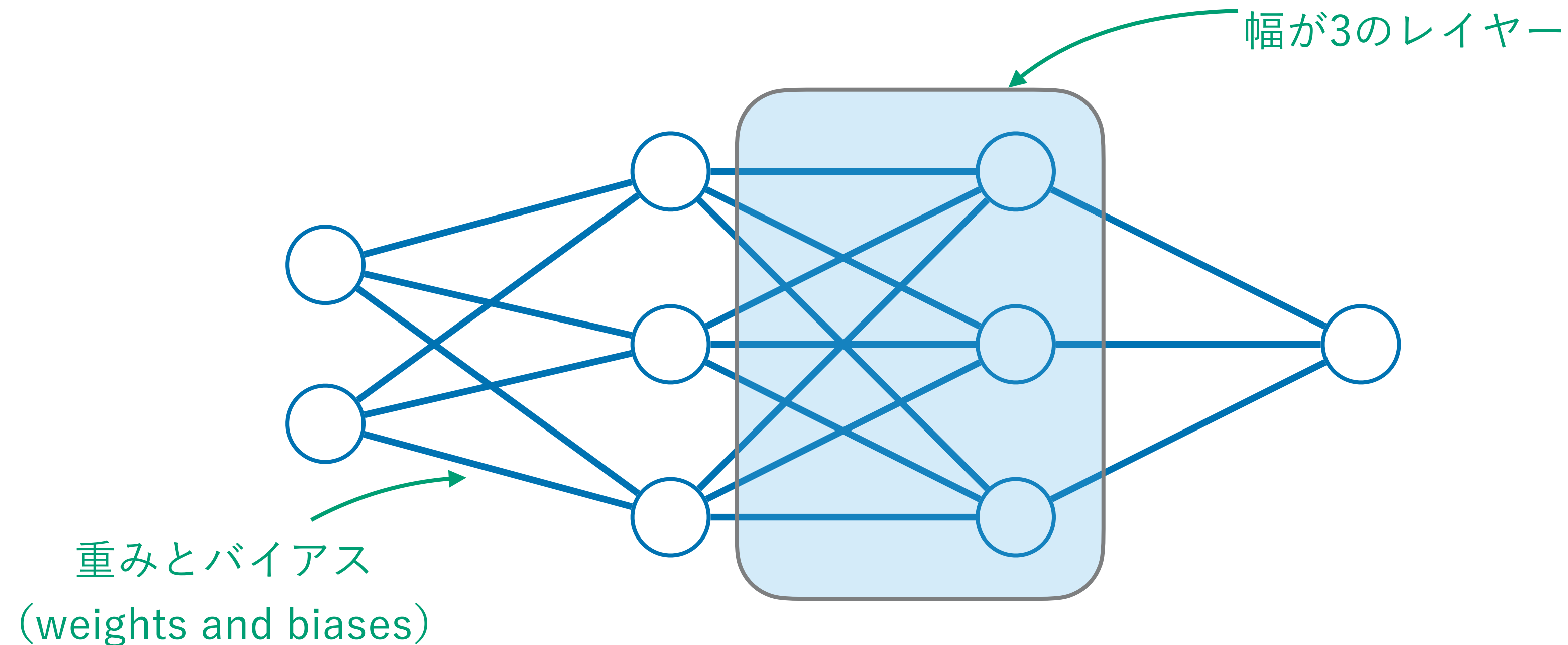
受け取った値から
計算された活性値が
保持される

- ユニットが保持する「現在の値」がまた後続のユニットに渡されていく

ニューラルネット＞表現図式と用語＞用語

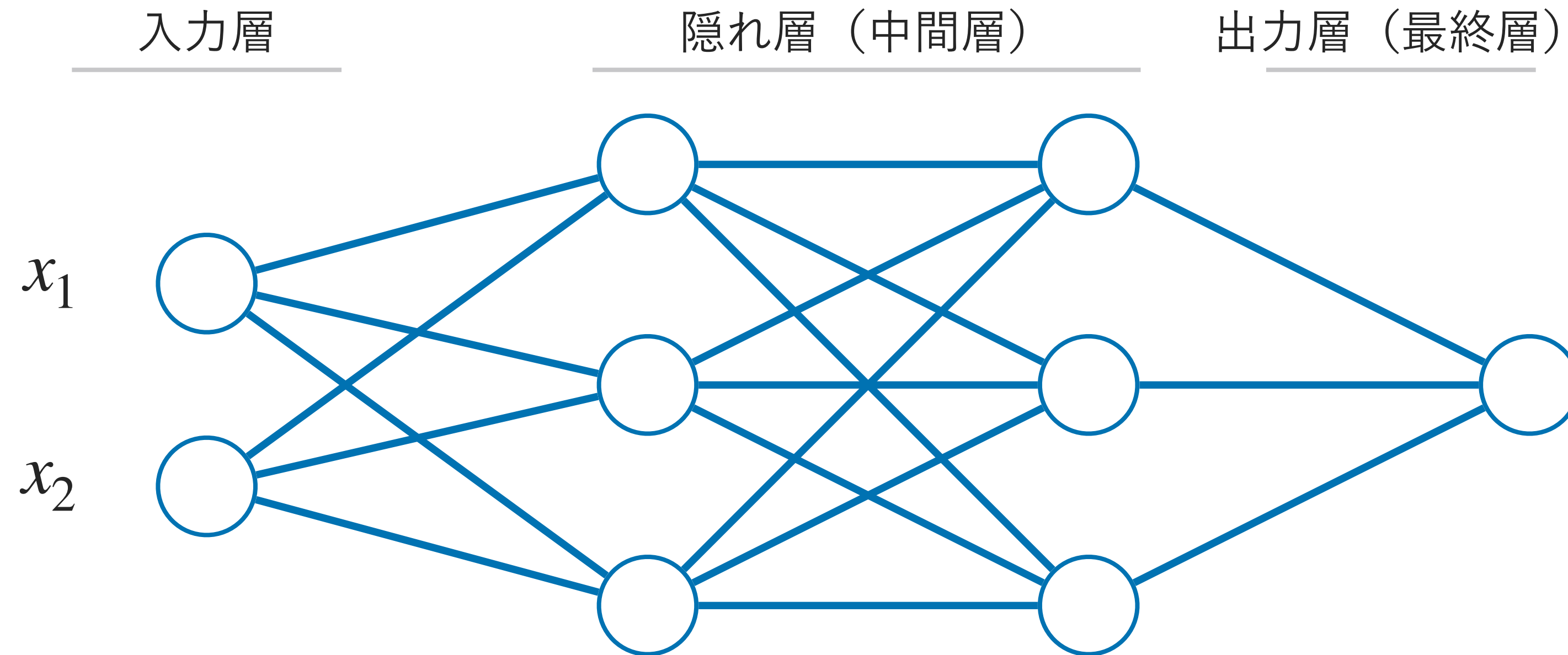
ユニットを複数並べたものがレイヤーとなる

- 並べた個数はレイヤーの幅 (width) という



ニューラルネット＞表現図式と用語＞全体

ユニットで表した順伝播型ニューラルネットワーク（よく使われる図式）



隠れ層が2層ある → 隠れ2層のFNN
(two-hidden-layer FNN) という

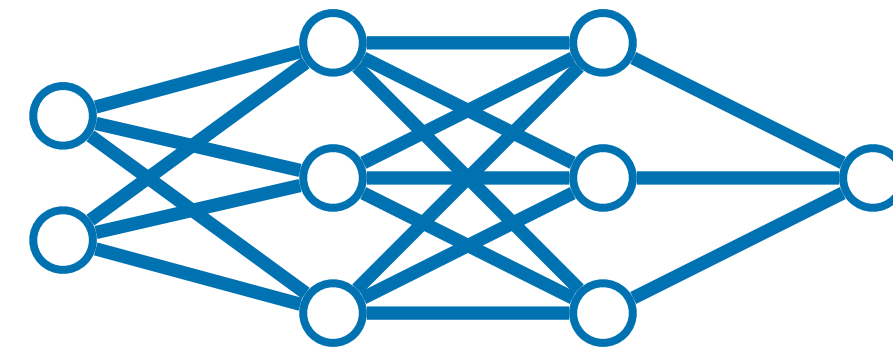
入力層を除いて3層ある → 3層FNN
(three-layer FNN) ともいう

ニューラルネット > アーキテクチャー



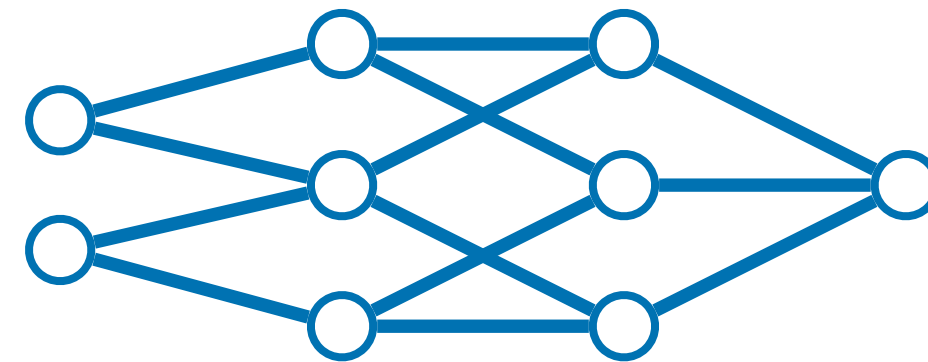
ユニット単位で見れば，結合の仕方はもっと自由に設計してもよい

- 今回扱ったレイヤーは，前の層の**全ての**ユニットから，今の層の**全ての**ユニットに線が繋がっているので，**全結合層（fully-connected layer）**と呼ばれる



全結合

- 例えば，次のようなネットワークを考えてもよい



あえて一部だけ結合

ニューラルネット > まとめ > モデルの構造



ニューラルネットワークの人気の理由の一つは、その柔軟さにある

- ネットワークの設計のパターンは**アーキテクチャ (architecture)** と呼ばれる
- 応用分野ごとに、問題に合った形のモデルを生み出すことができる
 - 画像に適した構造, テキストに適した構造, ……が研究・開発されている
- 応用分野ごとの適したアーキテクチャ設計は、学習における事前知識の典型例

ニューラルネット > 補足 > 活性化関数の役割



活性化関数は省いてはいけない？

- 活性化関数がないければ、いくつ層を重ねても、結局1つの層に縮約できてしまう

$$W_2(W_1x + b_1) + b_2 = (W_2W_1)x + (W_2b_1 + b_2)$$

- つまり、活性化関数が無ければ、1層分の複雑さの関数しか表すことができない
- 線型（+ 定数）よりも複雑な関数を表せるようにするのが活性化関数の役割
 - ゆえに、活性化関数のことを「非線型性」（nonlinearity）と呼ぶこともある

ニューラルネット > 補足 > ユニットとレイヤー

ユニットとレイヤーの対応

- 重み行列 W , バイアス b , 活性化関数 ψ からなるレイヤーなら

$$W = \begin{bmatrix} \mathbf{w}_1^\top \\ \vdots \\ \mathbf{w}_r^\top \end{bmatrix}, \quad b = \begin{bmatrix} b_1 \\ \vdots \\ b_r \end{bmatrix} \quad \text{とすると,} \quad \psi(W\mathbf{x} + b) = \begin{bmatrix} \psi(\mathbf{w}_1^\top \mathbf{x} + b_1) \\ \vdots \\ \psi(\mathbf{w}_r^\top \mathbf{x} + b_r) \end{bmatrix}$$

- これは, 次のユニット (unit, 素子) を r 個並べたものともみなせる
 - w を重み (weight), b をバイアス (bias), $\psi: \mathbb{R} \rightarrow \mathbb{R}$ を活性化関数として

$$\mathbf{x} \mapsto \psi(\mathbf{w}^\top \mathbf{x} + b)$$

【用語】 「重み」は $\mathbf{w}^\top \mathbf{x} = w_1 x_1 + \dots + w_p x_p$ という重み付き和の重みであることから. 「バイアス (偏り)」は, 出力の値を $+b$ だけ「ずらす」ことから.